

Grocery Store Management System

DBMS Project Documentation

Academic Project Coursework

This documentation serves as the comprehensive guide, requirements analysis, database schema definition, and system architecture report for the **Grocery Store Management System**.

1. Problem Definition

1.1 Background Context

Traditional small-to-medium-scale grocery stores often rely on manual ledgers or fragmented spreadsheet-based systems to record inventory, track sales, and process customer checkouts. This manual approach is highly error-prone, inefficient, and limits the shopkeeper's ability to respond quickly to market demands.

1.2 Identified Pain Points

- **Inefficient Inventory Control:** Without real-time tracking, shopkeepers cannot easily determine current stock levels. This leads to either stockouts (losing customers) or overstocking (tying up capital).
- **High Expiry Losses:** Fresh products, dairy, and canned items have strict shelf lives. A lack of expiry date tracking results in expired goods sitting on shelves or being discarded, directly hurting profitability.
- **Manual Billing Slowdowns:** Calculating checkout totals manually or searching through static lists slows down the point-of-sale (POS) process, creating long queues and frustrating customers.
- **No Sales Insights:** Without a centralized database logging every sale, the business owner has no visibility into peak sales hours, fast-moving items, or daily revenue trends.
- **Lack of Data Integrity:** Manual ledger entry often leads to inconsistent records (e.g., selling a product that is out of stock, recording mismatched product names, or incorrect tax/pricing calculations).

1.3 The Solution: Grocery Store Management System

The proposed system is an integrated, database-driven web application comprising a **React frontend**, a **Spring Boot REST API backend**, and a **MariaDB relational database**. It solves the manual problems by:

1. **Automating Inventory Management:** Providing a real-time CRUD (Create, Read, Update, Delete) interface with automatic low-stock highlighting.
2. **Point-of-Sale (POS) Integration:** Simulating a checkout counter where items are added to a cart, stock levels are automatically decremented upon purchase, and transactions are permanently archived.
3. **Data Integrity & Consistency:** Enforcing rigid foreign key constraints, check constraints, and cascade deletes to guarantee that the database remains in a consistent state at all times.

2. Data Collection (Database Schema & Dictionary)

The database schema, initialized in [schema.sql](#), consists of four heavily relational tables. Below is the comprehensive data dictionary detailing each entity, its attributes, types, and constraints.

2.1 Table: `categories`

Stores the categorization of products to structure the inventory logically.

Field Name	Data Type	Key	Null	Default / Extra	Description & Constraints
id	INT	PK	NO	AUTO_INCREMENT	Unique identifier for each category.
name	VARCHAR(50)	UNI	NO	None	Descriptive category name (e.g., 'Bakery', 'Beverages'). Must be unique.

2.2 Table: products

Maintains the master list of all grocery items available in the store.

Field Name	Data Type	Key	Null	Default / Extra	Description & Constraints
id	INT	PK	NO	AUTO_INCREMENT	Unique identifier for the product.
name	VARCHAR(100)	-	NO	None	Display name of the product.
sku	VARCHAR(50)	UNI	NO	None	Stock Keeping Unit code for barcode/fast scanning. Must be unique.
category_id	INT	FK	YES	NULL	References categories(id). Set to NULL on category deletion.
price	DECIMAL(10, 2)	-	NO	None	Sales price per unit. Must be \$>= 0.00\$.
quantity	INT	-	NO	0	Available stock in the store. Must be \$>= 0\$.
unit	VARCHAR(20)	-	NO	None	Unit of measurement (e.g., 'kg', 'pcs', 'packet', 'liter').
description	TEXT	-	YES	NULL	Optional descriptive notes about the product.
expiry_date	DATE	-	YES	NULL	Expiration date of the batch.
min_stock_level	INT	-	NO	5	Threshold for reordering. Generates alert when quantity <= min_stock_level. Must be \$>= 0\$.

2.3 Table: orders

Represents customer checkout sales receipts.

Field Name	Data Type	Key	Null	Default / Extra	Description & Constraints
------------	-----------	-----	------	-----------------	---------------------------

id	INT	PK	NO	AUTO_INCREMENT	Unique order/transaction receipt ID.
order_date	TIMESTAMP	-	NO	CURRENT_TIMESTAMP	Date and time the purchase was finalized.
total_amount	DECIMAL(10, 2)	-	NO	None	Sum of all line item costs. Must be \$>= 0.00\$.
payment_method	VARCHAR(20)	-	NO	None	Transaction payment mechanism. Enforced to: CASH, CARD, or UPI.

2.4 Table: `order_items`

Bridge entity representing specific products and quantities purchased under an order.

Field Name	Data Type	Key	Null	Default / Extra	Description & Constraints
id	INT	PK	NO	AUTO_INCREMENT	Unique row identifier for the transaction line item.
order_id	INT	FK	NO	None	References <code>orders(id)</code> . Deleted automatically on order cascade deletion.
product_id	INT	FK	YES	NULL	References <code>products(id)</code> . Set to NULL if product is discontinued.
quantity	INT	-	NO	None	Amount purchased in this order. Must be \$> 0\$.
unit_price	DECIMAL(10, 2)	-	NO	None	Historical unit price at the time of purchase. Must be \$>= 0.00\$.

3. System Requirements Analysis

3.1 Functional Requirements (FR)

The application fulfills the following operations categorized by module:

A. Administrative Dashboard

- **Real-time Overview Metrics:** Display total active revenue, active product count, and total orders processed.
- **Low-Stock Alerts:** Automatically scans inventory to display warning cards for products whose current `quantity` is less than or equal to their `min_stock_level`.

B. Inventory Management

- **Add/Edit Products:** Enable store admins to register new products with SKU, categories, prices, stock level, units, and expiration details.

- **Delete Products:** Discontinue products safely. Corresponding sales transaction records (`order_items`) will retain historical prices but reference `product_id` as `NULL` to preserve financial history.
- **Search & Filter:** Search products dynamically by name/SKU and filter them by category.

C. Category Management

- **Create/Delete Categories:** Organize products. Deleting a category does not delete its products; it sets the `category_id` to `NULL` to keep the inventory intact.

D. Point of Sale (POS) Billing

- **Interactive Checkout Interface:** Add items from the active inventory catalog to a checkout cart.
- **Stock Checks:** Verify quantity constraints before submitting checkouts. Prevent transaction submission if requested quantity exceeds available inventory stock.
- **Real-time Database Decrement:** Atomically decrease the product's quantity in the database upon checkout completion.
- **Payment Log:** Record totals and log payment methods (CASH/CARD/UPI).

E. Transaction History

- **Auditing:** Display a chronological list of all sales invoices, complete with receipt ID, date, total amount, payment channel, and itemized sub-totals.

3.2 Non-Functional Requirements (NFR)

- **Data Integrity:** MariaDB transaction isolation ensures billing cart items are committed together. Spring Boot `@Transactional` annotations prevent partial updates in case of backend server crashes during checkout.
- **Usability:** High-performance local caching using React state management renders database updates instantly.
- **Scalability:** Relational index optimizations on primary and foreign keys guarantee millisecond response times.

4. Entity-Relationship (ER) Diagram

4.1 Logical ER Diagram (Mermaid)

Below is the logical structure mapping relationships and cardinality (e.g., \$1\$ to many \$N\$, nullable relationships):

```
erDiagram
    CATEGORIES ||--o{ PRODUCTS : "has (1:N, Nullable)"
    ORDERS ||--|{ ORDER_ITEMS : "contains (1:N, Cascade)"
    PRODUCTS ||--o{ ORDER_ITEMS : "purchased_in (1:N, Nullable)"

    CATEGORIES {
        int id PK
        varchar name UK
    }

    PRODUCTS {
        int id PK
        varchar name
        varchar sku UK
```

```
    int category_id FK
    decimal price
    int quantity
    varchar unit
    text description
    date expiry_date
    int min_stock_level
}

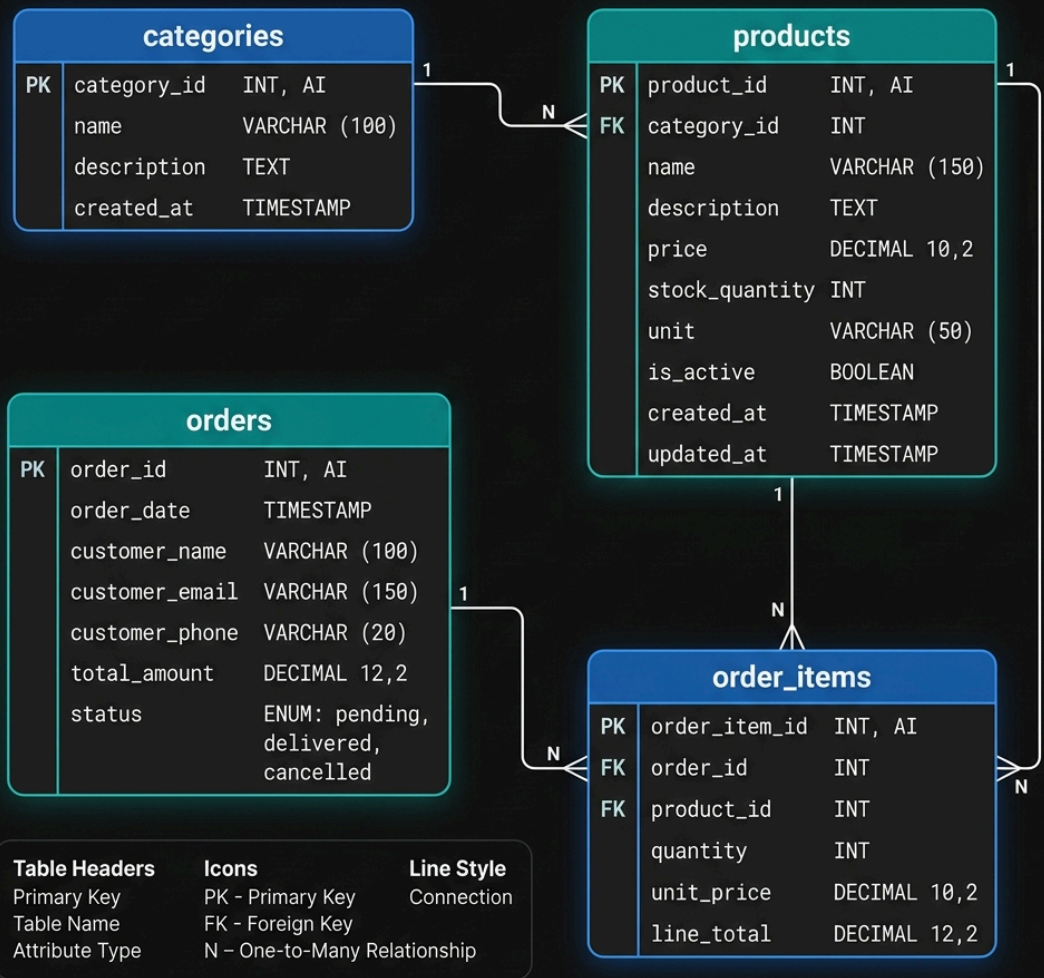
ORDERS {
    int id PK
    timestamp order_date
    decimal total_amount
    varchar payment_method
}

ORDER_ITEMS {
    int id PK
    int order_id FK
    int product_id FK
    int quantity
    decimal unit_price
}
```

4.2 Conceptual Database Diagram Image

The conceptual database diagram below provides a visual blueprint of the relationships:

Grocery Store Management System: Entity-Relationship Diagram (ERD)



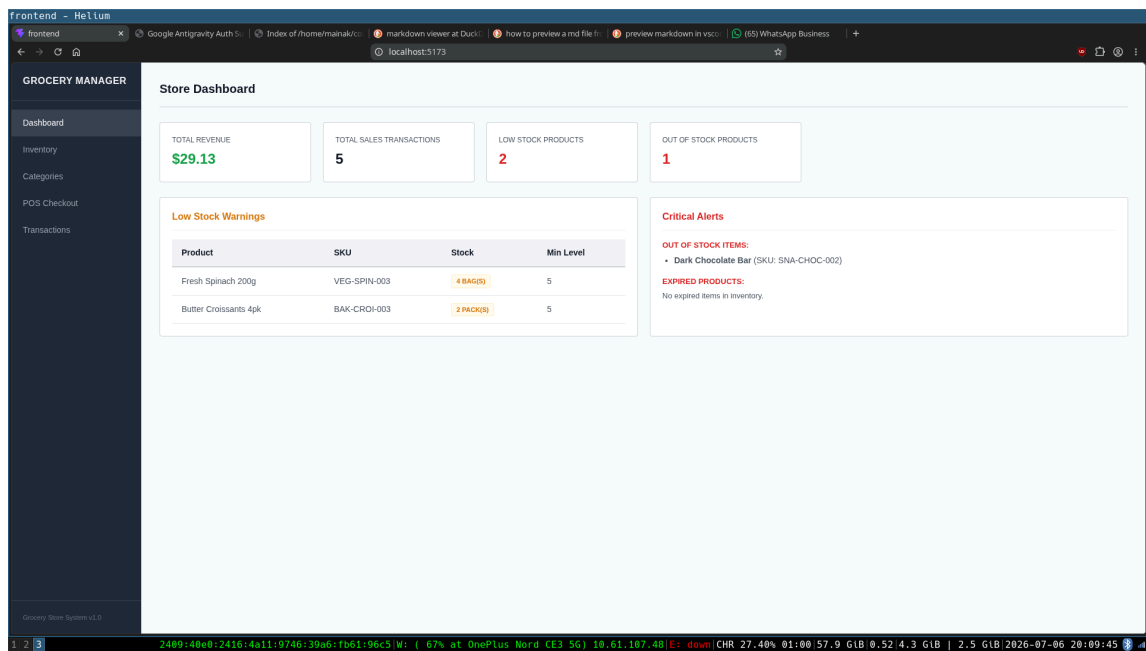
5. System Screenshots (User Interface Walkthrough)

[!TIP] To add your screenshots to this document:

Run your application locally (`npm run dev` inside frontend, and run your Maven Spring Boot backend).
Capture screenshots of the following pages and save them to the directory `docs/screenshots/` with the exact filenames listed below. The images will automatically load and replace the placeholders.

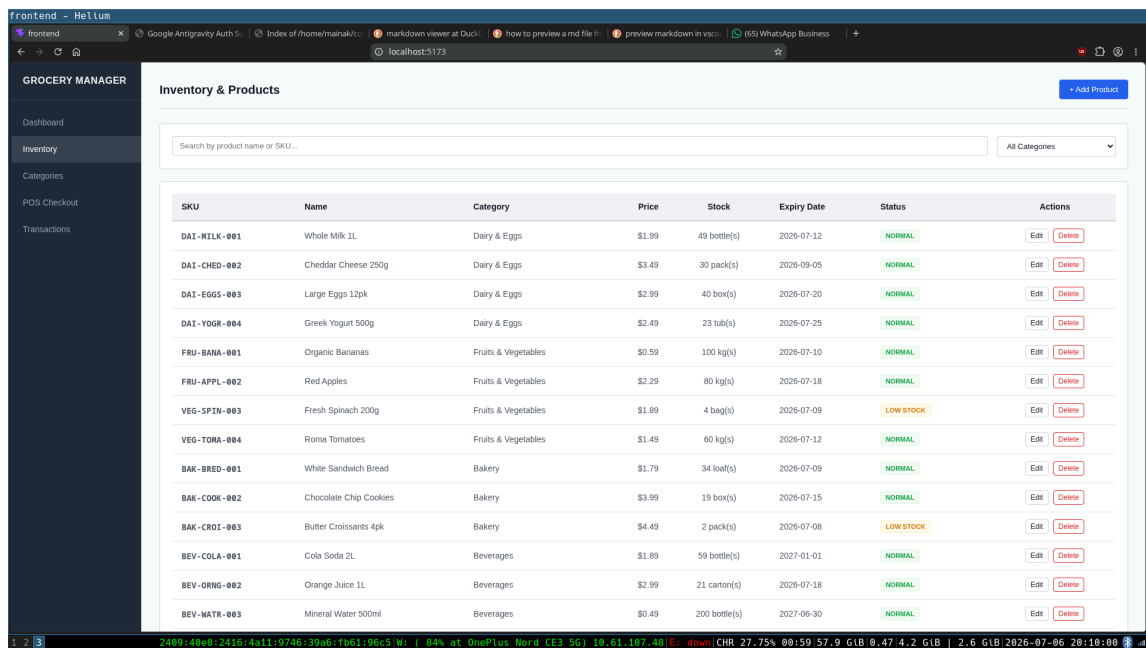
5.1 Dashboard View

Shows revenue counters, active product counts, completed order summaries, and real-time low-stock alerts.



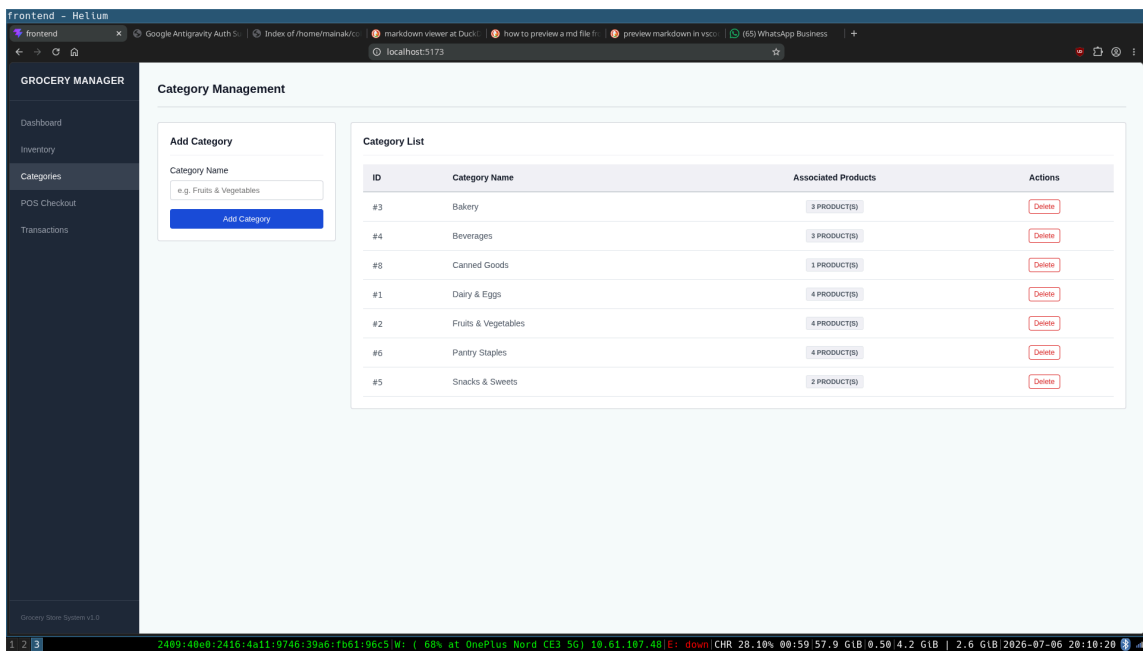
5.2 Inventory & Product Management

Interface for performing CRUD operations on products, viewing stock numbers, pricing, units, and editing items.



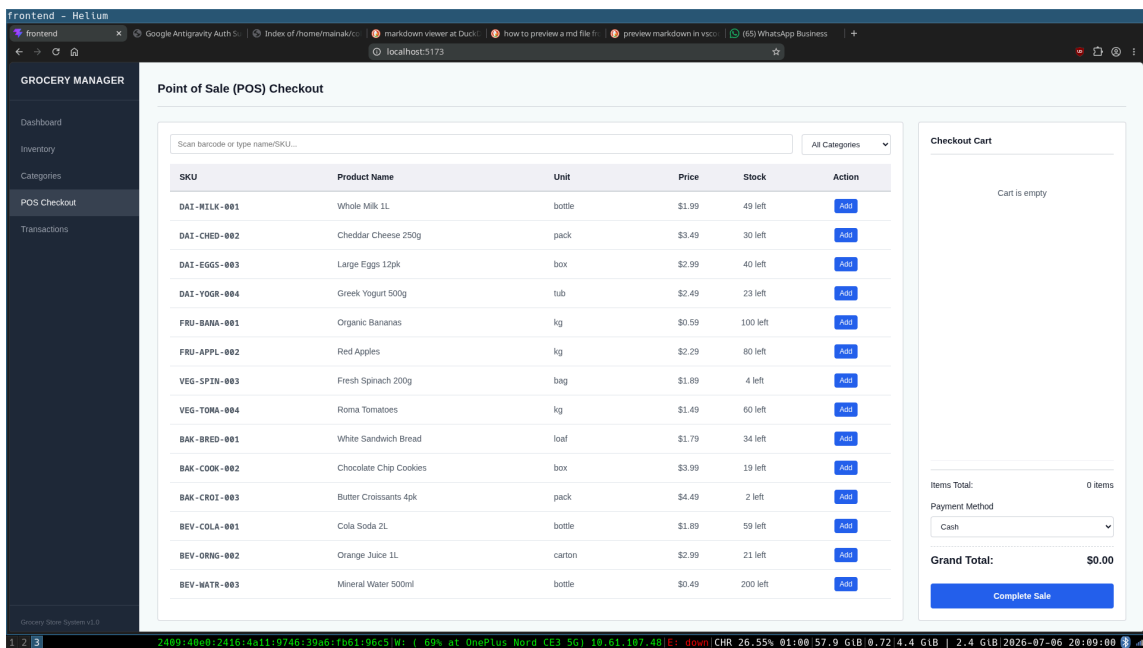
5.3 Category Management

Panel to add and delete product categories and view how many items belong to each category.



5.4 Point-of-Sale (POS) Checkout Page

Shopping cart layout where operators select items, enter quantity, check stock, and finalize payments.



5.5 Sales Transaction History

Chronological listing of invoices for auditing and record-keeping.

